

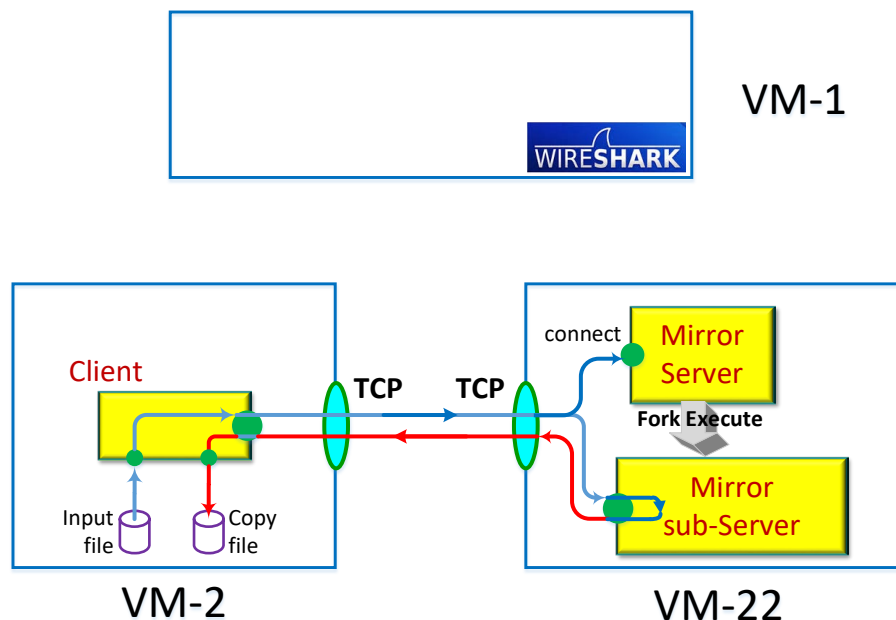
P4: Socket Programming Concurrent Servers

Learning Objectives:

- Properly exchange streams of data over TCP connections taking into account buffering issues and the possibility of operation systems signals arriving during slow system calls.
- Build a concurrent TCP server environment where a listening TCP server launches concurrent sub-servers to handle clients.
- Build a client that communicates with multiple servers.
- Contrast the operation of TCP vs. UDP client/server software.

Phase One:

You are to build a TCP “Client” on VM2 that establishes a connection with a TCP “Mirror Server” running indefinitely on VM22. When the “Mirror Server” establishes a connection with a client, it forks / executes a “Mirror sub-Server” in order to handle the new connection from the “Client”. As shown in the diagram, the client simply transmits the content of some file to the Mirror Server. The Mirror sub-Server merely echoes the same data it receives back to the client, who saves it into a separate file. It is possible that multiple clients simultaneously connect to the “Mirror Server”, thus resulting in multiple “Mirror sub-Server” processes being concurrently running on VM22. Wireshark should be running as a super user on VM1 to capture the traffic between VM2 and VM22. Restrict you displayed packets to only ones involving the MIRROR_TCP_PORT. This set-up is illustrated in the following diagram:

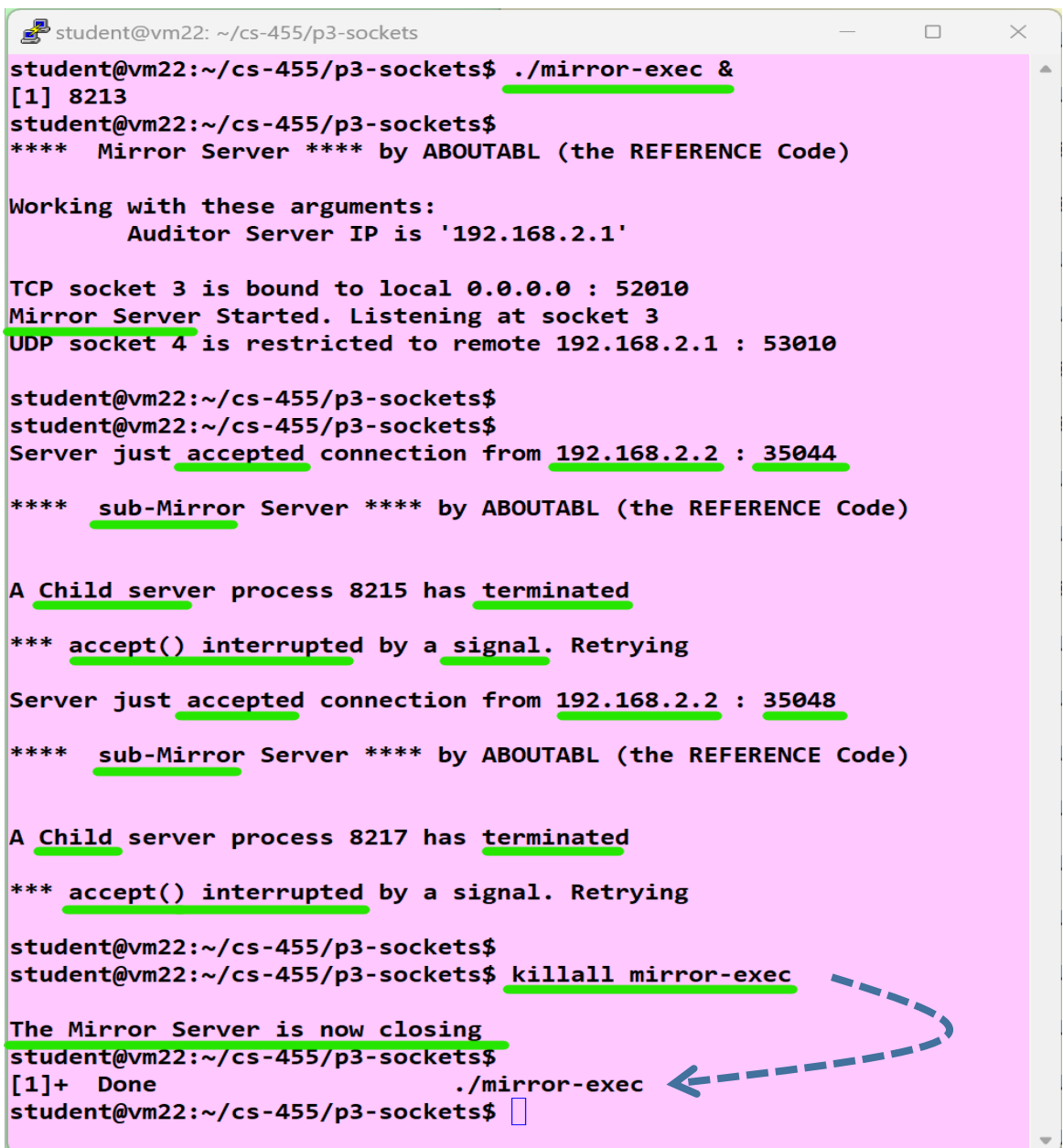


Due to TCP’s tendency to partition the transmitted data into multiple segments, it is vital that you accommodate such behavior by making as many read attempts as needed to receive the entire data stream you expect. Moreover, the sender of TCP data may have to make several attempts to send the entire amount of data back to the client. This usually occurs when the local TCP send-window has no more room due to network congestion and/or slow receiver. While being blocked inside any slow system call, e.g. `read()`, `accept()`, `write()`, ... etc., if a signal unblocks the call, you must re-start the interrupted system call.

Phase One: The Mirror Server & sub-Mirror Server

The Mirror Server, referred to as the “Mirror” from now on, starts by creating a TCP socket bound to the local port specified by ‘MIRROR_TCP_PORT’. The Mirror must also prepare to catch the kill signal “SIGTERM” and any signals indicating change of child process status “SIGCHLD”. Next, it starts listening at that port and enters an indefinite loop where:

- it waits for a remote client to connect.
 - it forks / executes a child process running the “subMirror” code. Remember to pass the necessary socket descriptors to the subMirror as command-line arguments.
 - Remember to close any sockets unnecessary to the operation of either the parent or the child processes.
- The Mirror sub-Server, referred to as the “subMirror” learns the connected socket descriptor to the client from its parent Mirror via command-line arguments. It can perform any arbitrary logic that is relevant to your application. For this assignment, the subMirror simply echoes the data received from the client at the connected socket back to that client on the same socket. Here is a screenshot showing the Mirror Server at run time:



```
student@vm22: ~/cs-455/p3-sockets
student@vm22:~/cs-455/p3-sockets$ ./mirror-exec &
[1] 8213
student@vm22:~/cs-455/p3-sockets$
**** Mirror Server **** by ABOUTABL (the REFERENCE Code)

Working with these arguments:
    Auditor Server IP is '192.168.2.1'

TCP socket 3 is bound to local 0.0.0.0 : 52010
Mirror Server Started. Listening at socket 3
UDP socket 4 is restricted to remote 192.168.2.1 : 53010

student@vm22:~/cs-455/p3-sockets$
student@vm22:~/cs-455/p3-sockets$
Server just accepted connection from 192.168.2.2 : 35044

**** sub-Mirror Server **** by ABOUTABL (the REFERENCE Code)

A Child server process 8215 has terminated
*** accept() interrupted by a signal. Retrying
Server just accepted connection from 192.168.2.2 : 35048

**** sub-Mirror Server **** by ABOUTABL (the REFERENCE Code)

A Child server process 8217 has terminated
*** accept() interrupted by a signal. Retrying

student@vm22:~/cs-455/p3-sockets$
student@vm22:~/cs-455/p3-sockets$ killall mirror-exec

The Mirror Server is now closing
student@vm22:~/cs-455/p3-sockets$
[1]+ Done ./mirror-exec
student@vm22:~/cs-455/p3-sockets$
```

Phase One: The Client

For this assignment, the Client simply opens an existing file, sends the content of that file via a TCP connection to the Mirror Server, receives a replica of the file back from the Mirror Server, and saves it to a new file that shares the same name as the input file, but with a **.copy** extension. Here is a screenshot showing the Client at run time:

```
student@vm2: ~/cs-455/p3-sockets
student@vm2:~/cs-455/p3-sockets$
student@vm2:~/cs-455/p3-sockets$ ./client

**** Client **** by ABOUTABL (the REFERENCE Code)

Working with these arguments:
    Input   File Name is 'GoldilockAnd3Bears.txt'
    Mirror  Server IP is '192.168.2.22'
    Auditor Server IP is '192.168.2.1'

TCP socket 5 is connected to remote 192.168.2.22 : 52010
TCP Client is now connected to the TCP Mirror server 192.168.2.22 : 52010
UDP socket 6 is restricted to remote 192.168.2.1 : 53010
TCP Client finished sending the local file to the TCP Mirror server

TCP Client closed the connection to the TCP Mirror server

student@vm2:~/cs-455/p3-sockets$
student@vm2:~/cs-455/p3-sockets$ ./client 3LittleWolvesAndBigBadPig.txt

**** Client **** by ABOUTABL (the REFERENCE Code)

Working with these arguments:
    Input   File Name is '3LittleWolvesAndBigBadPig.txt'
    Mirror  Server IP is '192.168.2.22'
    Auditor Server IP is '192.168.2.1'

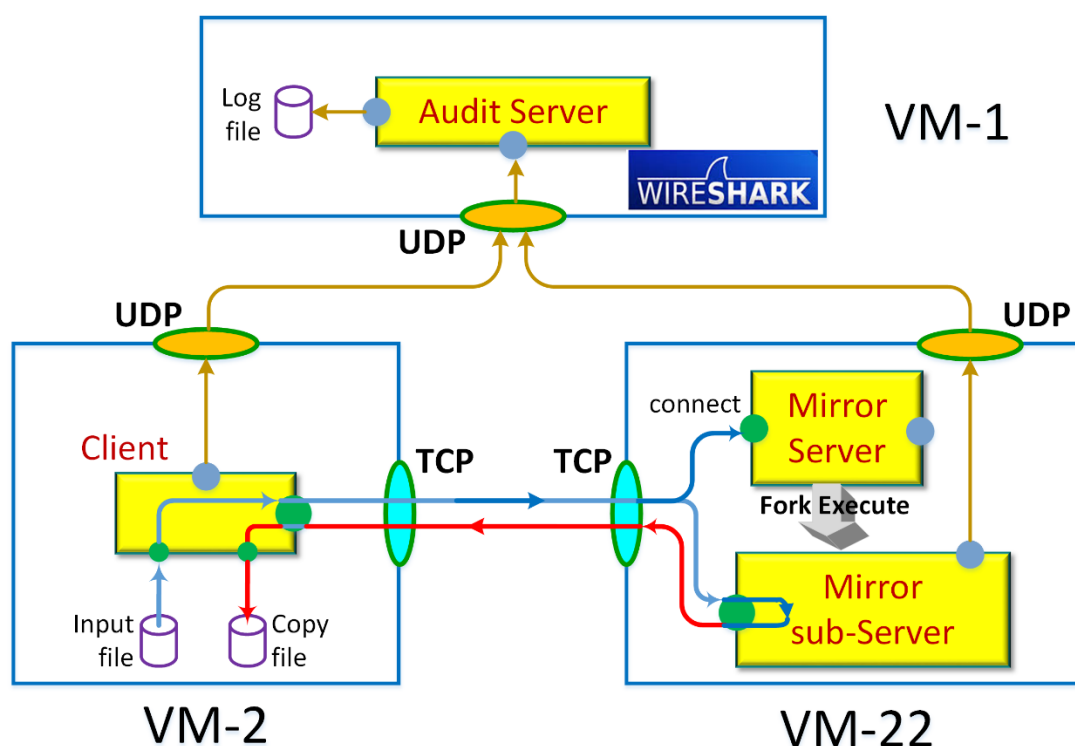
TCP socket 5 is connected to remote 192.168.2.22 : 52010
TCP Client is now connected to the TCP Mirror server 192.168.2.22 : 52010
UDP socket 6 is restricted to remote 192.168.2.1 : 53010
TCP Client finished sending the local file to the TCP Mirror server

TCP Client closed the connection to the TCP Mirror server

student@vm2:~/cs-455/p3-sockets$
student@vm2:~/cs-455/p3-sockets$
```

Phase Two: The Auditor

The Auditor is a UDP server application which receives and logs (both to a log file, and to the **stdout** device) any send / receive transactions done at the Client and at the sub-Mirror Server. Each reported transaction consists of three pieces of information: a transaction operation code { **1→send** vs. **2→receive** }, the size of the transaction, i.e. the number of sent or received bytes, and the 32-bit IP address of the remote peer with which this transaction is done. The Auditor learns the owner of the transaction as a side effect of using the **recvfrom()** system call.



The Auditor Server continuously waits at the `AUDITOR_UDP_PORT` for any incoming UDP datagrams sent by the Client or the sub-Mirror Server. The Auditor Server logs those same transactions to a text log file. It also displays the details of those datagrams on the **stdout** device as illustrated in the screenshot of a sample run shown on the next page. You need to add the appropriate code to the Client, and the sub-Mirror Server to send a UDP datagram carrying a transaction report to the Auditor each time a chunk of data is sent or received by both the Client and the sub-Mirror Server.

```
student@vm1: ~/cs-455/p3-sockets
student@vm1:~/cs-455/p3-sockets$ ./auditor &
[1] 6391
student@vm1:~/cs-455/p3-sockets$
**** Auditor Server **** by ABOUTABL (the REFERENCE Code)

UDP socket 4 is bound to local 0.0.0.0 : 53010

### **** Auditor Server **** by ABOUTABL (the REFERENCE Code) has started

student@vm1:~/cs-455/p3-sockets$
student@vm1:~/cs-455/p3-sockets$
### Activity By 192.168.2.2 :33887 ... Sent 1000 Bytes. Peer's IP: 192.168.2.22
### Activity By 192.168.2.22 :49719 ... Received 1000 Bytes. Peer's IP: 192.168.2.22
### Activity By 192.168.2.22 :49719 ... Sent 1000 Bytes. Peer's IP: 192.168.2.22
### Activity By 192.168.2.2 :33887 ... Received 1000 Bytes. Peer's IP: 192.168.2.22
### Activity By 192.168.2.2 :33887 ... Sent 1000 Bytes. Peer's IP: 192.168.2.22
### Activity By 192.168.2.22 :49719 ... Received 1000 Bytes. Peer's IP: 192.168.2.22
### Activity By 192.168.2.22 :49719 ... Sent 1000 Bytes. Peer's IP: 192.168.2.22
### Activity By 192.168.2.2 :33887 ... Received 1000 Bytes. Peer's IP: 192.168.2.22
### Activity By 192.168.2.2 :33887 ... Sent 402 Bytes. Peer's IP: 192.168.2.22
### Activity By 192.168.2.22 :49719 ... Received 402 Bytes. Peer's IP: 192.168.2.22
### Activity By 192.168.2.22 :49719 ... Sent 402 Bytes. Peer's IP: 192.168.2.22
### Activity By 192.168.2.2 :33887 ... Received 402 Bytes. Peer's IP: 192.168.2.22
### Activity By 192.168.2.2 :36516 ... Sent 1000 Bytes. Peer's IP: 192.168.2.22
### Activity By 192.168.2.22 :49719 ... Received 1000 Bytes. Peer's IP: 192.168.2.22
### Activity By 192.168.2.22 :49719 ... Sent 1000 Bytes. Peer's IP: 192.168.2.22
### Activity By 192.168.2.2 :36516 ... Received 1000 Bytes. Peer's IP: 192.168.2.22
### Activity By 192.168.2.2 :36516 ... Sent 1000 Bytes. Peer's IP: 192.168.2.22
### Activity By 192.168.2.22 :49719 ... Received 1000 Bytes. Peer's IP: 192.168.2.22
### Activity By 192.168.2.22 :49719 ... Sent 1000 Bytes. Peer's IP: 192.168.2.22
### Activity By 192.168.2.2 :36516 ... Received 1000 Bytes. Peer's IP: 192.168.2.22
### Activity By 192.168.2.2 :36516 ... Sent 489 Bytes. Peer's IP: 192.168.2.22
### Activity By 192.168.2.22 :49719 ... Received 489 Bytes. Peer's IP: 192.168.2.22
### Activity By 192.168.2.22 :49719 ... Sent 489 Bytes. Peer's IP: 192.168.2.22
### Activity By 192.168.2.2 :36516 ... Received 489 Bytes. Peer's IP: 192.168.2.22

student@vm1:~/cs-455/p3-sockets$
student@vm1:~/cs-455/p3-sockets$ killall auditor

Auditor is now closing
student@vm1:~/cs-455/p3-sockets$
```

Your Mission:

Guided by the skeleton code provided to you, create a **p4-sockets** folder inside which, you:

- Implement all the missing functions in **myNetLib.c** and declared in **myNetLib.h**
- Write the main programs: the Client in **client.c** , the Mirror Server in **mirror-exec.c** , the sub-Mirror Server process in **subMirror.c** , and the Auditor Server in **auditor.c** .
- Use the provided **makefile** to build all the executables for The Client, The Mirror server, the subMirror Server, and the Auditor Server.

What to Submit:

1. A non-zipped tar file of your **myNetLib.c** , **client.c** , **mirror-exec.c** , **subMirror.c** , and

```
tar -cvf name1_name2-p4.tar myNetLib.c client.c mirror-exec.c subMirror.c auditor.c
```

auditor.c source code files. Use this Linux command (with your full names):

2. Three screen shots (not inside the tar file) of three separate terminals: one for the Client, one for the Mirror Server, and one for the Auditor Server ...
3. The PCAP file (not inside the tar file) captured by Wireshark showing the traffic involving ONLY the ports of the two servers you created.